

StyleSense Algorithm Simulation & Pen-and-Paper Defense Guide

This guide is designed for the thesis/capstone **Algorithm Simulation & Testing Defense**. It maps your system's codebase, pipeline diagram, and mathematical calculations directly to the professor's 10-step rubric.

1. Why Data Preprocessing Matters to the Algorithm Simulation

In machine learning and pen-and-paper algorithm defense, **algorithms cannot compute anything on raw real-world data directly**:

1. **Computers & Math only understand numbers:** A machine or mathematical formula (like Euclidean distance or matrix convolution) cannot multiply or measure words like "TOP", "CASUAL", or "Blue".
2. **Preventing Scale Distortion:** Raw pixel values range from [0, 255]. Without scaling ($x / 255.0$ or Z-score standardization), large numbers cause exploding gradients or heavily skew Euclidean distance calculations.
3. **Dimensional Alignment (One-Hot Encoding):** Converting categorical features into binary vectors (\$0\$s and \$1\$s) allows algorithms like **K-Means Clustering** or **Cosine Similarity** to calculate distances in vector space:

$$d(p,q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

2. Codebase Audit vs. Professor's Consultation List

#	Professor's Consultation Step	Status in StyleSense Codebase	Pen-and-Paper Defense Strategy
1	Dataset Search	Implemented: <code>ml-service/export_dataset.py</code> , Admin manifest API.	Reference fashion datasets (DeepFashion / StyleSense Wardrobe Storage).
2	How to clean & preprocess	Implemented: PyTorch transforms (<code>Resize(224)</code> , <code>ToTensor</code> , <code>Normalize</code>) in <code>ml-service/train.py</code> .	Explain Min-Max normalization for image pixels and One-Hot Encoding for wardrobe attributes.
3	8 Rows taken from dataset	Implemented in Simulation Kit: <code>ml-service/simulation_kit.py</code> extracts 8 representative wardrobe items (W01–W08).	Present the 8-row table with 6 feature columns.
4	Train/Validation Split (80-20 or 70-30)	Implemented: <code>--val-split 0.20</code> in <code>ml-service/train.py</code> .	Split 8 rows into 6 Training rows (80%) and 2 Testing rows (20%) .
5	Identify outcome factors (Features)	Implemented: Category, Color, Style, Occasion.	Features: $x = [is_TOP, is_BOTTOM, is_SHOES, is_OUTER, is_FORMAL, is_NEUTRAL]$; Target: Style cluster / Category.
6	Data Cleaning	Implemented: Removal of DB IDs, timestamps, and redundant metadata.	Drop <code>id</code> , <code>clothing_name</code> , and <code>laundry_status</code> from the mathematical vector.
7	Convert decisions to 1s and 0s	Implemented: One-Hot / Binary vectorization in <code>simulation_kit.py</code> .	Form 6-dimensional vectors for all 8 rows.
8	2-fold / Train-Test Split	Implemented: 6 Train items, 2 Test items.	Calculate cluster centroids on Train items, evaluate on Test items.
9	Simulate algorithm using equations	Implemented: <code>simulation_kit.py</code> runs exact equations.	Step-by-step arithmetic for Euclidean Distance + Centroid Updates + CNN Convolution & Softmax.
10	Test simulation & Model accuracy	Implemented: Evaluates on 20% test items (W07, W08).	Accuracy = $\frac{2}{2} = 100\%$ on test sample.

3. Pen-and-Paper Simulation #1: K-Means Clustering (8-Row Tabular Test)

Step A: 8-Row Sample & One-Hot Preprocessing

ID	Item Name	Category	Style	Color	Vector $[x_1..x_6]$	Split
W01	White Oxford Shirt	TOP	FORMAL	White	$[1, 0, 0, 0, 1, 1]$	Train (80%)
W02	Graphic Band Tee	TOP	CASUAL	Black	$[1, 0, 0, 0, 0, 1]$	Train (80%)
W03	Blue Slim Chinos	BOTTOM	FORMAL	Blue	$[0, 1, 0, 0, 1, 0]$	Train (80%)
W04	Black Cargo Joggers	BOTTOM	CASUAL	Black	$[0, 1, 0, 0, 0, 1]$	Train (80%)
W05	Leather Derby Shoes	SHOES	FORMAL	Brown	$[0, 0, 1, 0, 1, 0]$	Train (80%)
W06	Running Sneakers	SHOES	CASUAL	White	$[0, 0, 1, 0, 0, 1]$	Train (80%)
W07	Tailored Blazer	OUTERWEAR	FORMAL	Navy	$[0, 0, 0, 1, 1, 1]$	Test (20%)
W08	Oversized Hoodie	OUTERWEAR	CASUAL	Gray	$[0, 0, 0, 1, 0, 1]$	Test (20%)

Vector schema: $[is_TOP, is_BOTTOM, is_SHOES, is_OUTERWEAR, is_FORMAL, is_NEUTRAL]$

Step B: Training Phase (Centroid Initialization & Update)

1. Initial Centroids (Seeded from W01 and W02):

- $C_1 = [1, 0, 0, 0, 1, 1]$ (Formal Cluster Seed)
- $C_2 = [1, 0, 0, 0, 0, 1]$ (Casual Cluster Seed)

2. Distance Calculation to Assign Clusters:

$$d(x, C) = \sqrt{\sum_{i=1}^6 (x_i - C_i)^2}$$

- W01: $d(x, C_1) = 0.000, d(x, C_2) = 1.000 \implies$ Cluster 1
- W02: $d(x, C_1) = 1.000, d(x, C_2) = 0.000 \implies$ Cluster 2
- W03: $d(x, C_1) = 1.732, d(x, C_2) = 2.000 \implies$ Cluster 1
- W04: $d(x, C_1) = 1.732, d(x, C_2) = 1.414 \implies$ Cluster 2
- W05: $d(x, C_1) = 1.732, d(x, C_2) = 2.000 \implies$ Cluster 1
- W06: $d(x, C_1) = 1.732, d(x, C_2) = 1.414 \implies$ Cluster 2

Cluster 1 (Formal): {W01, W03, W05}
Cluster 2 (Casual): {W02, W04, W06}

3. Centroid Update:

$$C_k = \frac{1}{|S_k|} \sum_{x \in S_k} x$$

- $C_1 = \frac{1}{3} ([1, 0, 0, 0, 1, 1] + [0, 1, 0, 0, 1, 0] + [0, 0, 1, 0, 1, 0]) = [0.33, 0.33, 0.33, 0.00, 1.00, 0.33]$
- $C_2 = \frac{1}{3} ([1, 0, 0, 0, 0, 1] + [0, 1, 0, 0, 0, 1] + [0, 0, 1, 0, 0, 1]) = [0.33, 0.33, 0.33, 0.00, 0.00, 1.00]$

Step C: Testing Phase on 20% Unseen Data (W07, W08)

1. Test Item W07 (Tailored Blazer, actual = FORMAL):

- Vector: $x_{W07} = [0, 0, 0, 1, 1, 1]$
- $d(x, C_1) = \sqrt{(0 - 0.33)^2 + (0 - 0.33)^2 + (0 - 0.33)^2 + (1 - 0)^2 + (1 - 1)^2 + (1 - 0.33)^2} = \sqrt{1.777} = 1.333$
- $d(x, C_2) = \sqrt{(0 - 0.33)^2 + (0 - 0.33)^2 + (0 - 0.33)^2 + (1 - 0)^2 + (1 - 0)^2 + (1 - 1)^2} = \sqrt{2.327} = 1.525$
- Since $1.333 < 1.525$, predict **Cluster 1 (FORMAL)** $\rightarrow$ **CORRECT** ✓

2. Test Item W08 (Oversized Hoodie, actual = CASUAL):

- Vector: $x_{W08} = [0, 0, 0, 1, 0, 1]$
- $d(x, C_1) = \sqrt{0.11 + 0.11 + 0.11 + 1.0 + (0 - 1)^2 + (1 - 0.33)^2} = \sqrt{2.777} = 1.666$
- $d(x, C_2) = \sqrt{0.11 + 0.11 + 0.11 + 1.0 + (0 - 0)^2 + (1 - 1)^2} = \sqrt{1.327} = 1.152$
- Since $1.152 < 1.666$, predict **Cluster 2 (CASUAL)** $\rightarrow$ **CORRECT** ✓

3. Accuracy Calculation:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Test Items}} \times 100 = \frac{2}{2} = 100.0\%$$

4. Pen-and-Paper Simulation #2: CNN Image Classification

Step 1: Pixel Normalization

$$\text{Pixel}_{\text{norm}} = \frac{\text{Pixel}}{255.0}$$

Step 2: 2D Convolution $(I * K) + b$

For a 4×4 image patch and a 3×3 vertical edge filter $K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ with bias $b = 0.1$:

$$\text{Feature}(1, 1) = (0.78 \cdot 1) + (0.20 \cdot -1) + (0.75 \cdot 1) + (0.18 \cdot -1) + (0.08 \cdot 1) + (0.71 \cdot -1) + 0.1 = \mathbf{0.620}$$

Step 3: ReLU Activation

$$\text{ReLU}(z) = \max(0, z) \implies \text{Feature Map} = \begin{bmatrix} 0.62 & 0.82 \\ 0.00 & 0.00 \end{bmatrix}$$

Step 4: Max-Pooling (2×2)

$$\text{Pooled Value} = \max(0.62, 0.82, 0.00, 0.00) = \mathbf{0.820}$$

Step 5: Softmax Classification Probability

$$P(y = i \mid \mathbf{z}) = \frac{e^{z_i}}{\sum_{j=1}^5 e^{z_j}}$$

- $z(\text{TOP}) = (0.820 \cdot 1.8) + 0.1 = 1.576 \implies e^{1.576} = 4.835$
- $\sum e^z = 9.532$
- $P(\text{TOP}) = \frac{4.835}{9.532} = \mathbf{50.73\%}$ (Highest Probability $\implies$ Classification Output: **TOP**).

5. How to Run the Script

Run the automated simulation script anytime in your terminal:

```
python ml-service/simulation_kit.py
```